2.8 Интерфейсы

MERION

Интерфейс — это тип, который определяет набор методов, которые должны быть реализованы другими типами.

Интерфейсы позволяют абстрагироваться от реализации и работать с объектами через их поведение.

Характеристики интерфейсов

- **Упрощение кода**: Интерфейсы помогают разделять код на независимые блоки, взаимодействующие друг с другом через определенные методы
- **Утиная типизация**: В Go не требуется явно указывать, что тип реализует интерфейс. Если тип имеет все методы интерфейса, он автоматически удовлетворяет этому интерфейсу.
- **Гибкость**: Интерфейсы позволяют легко изменять и расширять код

Объявление интерфейса:

```
type Shape interface {
    Area() float64
}
```

Реализация интерфейса:

```
type Rectangle struct {
    Width  float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}
```

Использование интерфейсов:

```
package main

import "fmt"

type Shape interface {
    Area() float64
}

type Rectangle struct {
    Width  float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func PrintArea(s Shape) {
    fmt.Println("Площадь:", s.Area())
}

func main() {
    rect := Rectangle{Width: 10, Height: 5}
                                  Площадь: 50
```

MERION

Пустой интерфейс

```
package main

import "fmt"

func PrintValue(v interface{}) {
    fmt.Println("Значение:", v)
}

func main() {
    PrintValue(42)
    PrintValue("Hello Go!")
}
```

Метод-выбор (type switch): Можно использовать оператор выбора типа для определения конкретного типа переменной интерфейсного типа.

```
func Describe(i interface{}) {
    switch v := i.(type) {
    case int:
        fmt.Println("Это int:", v)
    case string:
        fmt.Println("Это string:", v)
    default:
        fmt.Println("Неизвестный тип")
    }
}
```

Type assertion: механизм в Go, который позволяет проверить, содержит ли переменная интерфейсного типа конкретный тип

```
package main

type Animal interface {
    Speak() string
}

type Dog struct{}
func (d Dog) Speak() string { return "Woof!" }

type Cat struct{}
func (c Cat) Speak() string { return "Meow!" }

func main() {
    var a Animal = Dog{}

    dog := a.(Dog)
    print(dog.Speak())

    if cat, ok := a.(Cat); ok {
        print(" Это кошка:", cat.Speak())
    } else {
        print(" Это не кошка!")
    }
```

```go
package main

func main() {
    var x = func() int { return 1 }
    x = nil
    test(x)
}

func test(x interface{}) {
    if x == nil {
        println("nil")
    }

    if x, ok := x.(func() int); ok {
        println("func=", x())
    }
}
```

Интерфейс на стороне получателя:

```go
package main

import "fmt"

type Logger interface {
    Info(string)
}

type Controller struct {
    logger Logger
}

func (c Controller) Handle(message string) {
    c.logger.Info(message)
}
```

Особенности работы с интерфейсами:

1. **Инициализация**: Интерфейсы в Go всегда инициализируются значением.
2. **Типы данных**: Интерфейс состоит из двух частей: типа (T) и значения (V).
3. **Минимальные интерфейсы**: Рекомендуется создавать небольшие интерфейсы (ISP)
